

Workforce backend

Technology & code guide

Prepared 25 September 2026. JavaScript backend for the supplied Workforce & Task Management PRD. This package is source code for you to run and deploy yourself. Nothing has been deployed.

1. What the backend contains

The API provides login and session refresh, three system roles, users, teams, projects, tasks, comments, work logs, status history, personal notifications, scoped dashboards/reports, and an Admin audit log. Separate workers send Nodemailer email, OneSignal push and live Socket.IO change signals using a durable PostgreSQL outbox and Redis queues. The daily overdue scheduler uses the same delivery path. The database schema, initial migration, fictional seed data, API specification, tests, and local container configuration are included.

The earlier frontend is a separate deliverable. Its demo API differs from this PRD. Use `FRONTEND-INTEGRATION.md` and `examples/frontend-client.ts` to connect it. Light/dark themes and Motion animations remain frontend concerns; the backend returns the same data for both themes.

2. Technology, location, and benefit

Versions below describe the implementation family. `package-lock.json` records the exact installed dependency versions; use `npm ci` to reproduce them.

Technology	Where it is used	What it does and why it helps
Node.js + JavaScript ES modules	<code>package.json</code> , <code>src/server.js</code> , <code>src/workers/index.js</code>	Runs the HTTP API and worker as separate processes; uses one language across the application. Node 22+ is required.
Express 5	<code>src/app.js</code> , <code>src/modules/**/*.route.js</code>	Maps HTTP requests to middleware and feature controllers; makes API modules independently understandable.
Prisma 6	<code>prisma/schema.prisma</code> , <code>src/config/prisma.js</code> , <code>repositories</code>	Represents tables, relations, indexes, and queries in one schema; parameterized queries reduce injection risk.
PostgreSQL / Supabase	<code>prisma/migrations/202609250001_initial/migration.sql</code>	Stores relational business data with foreign keys, unique constraints, and transactions. Supabase supplies hosted PostgreSQL when you configure it.
Zod 4	<code>src/config/env.js</code> , <code>src/modules/**/*.schema.js</code> , <code>src/middleware/validate.js</code>	Validates environment variables and request shapes before business logic; strict objects reject unexpected fields.
JWT / jsonwebtoken	<code>src/modules/auth/auth.token.js</code> , <code>src/middleware/authenticate.js</code>	Signs short-lived access tokens and checks algorithm, issuer, audience, expiry, and current database session.

Technology	Where it is used	What it does and why it helps
bcrypt	src/modules/auth/auth.service.js, src/modules/users/user.service.js , prisma/seed.js	Stores slow salted password hashes rather than readable passwords; default cost is 12 rounds.
Node crypto + refresh sessions	src/modules/auth/auth.token.js, auth.service.js, RefreshSession model	Generates high-entropy refresh tokens and stores their SHA-256 hashes; rotation and family revocation detect reuse.
Backend RBAC and scopes	src/middleware/authorize.js, feature services and routes	Checks both the action and the record: Admin globally, Manager managed teams, Employee assigned work. UI hiding cannot bypass these checks.
Optimistic concurrency	src/modules/tasks/task.service.js , src/modules/activities/activity. service.js	Updates only the expected task version; returns 409 instead of silently overwriting another person's change.
Serializable transactions	src/utils/transaction.js, mutation services	Commits task changes, history, notifications, and audit together; retries serialization conflicts up to three attempts.
Redis + ioredis	src/config/redis.js, src/middleware/security.js	Shares queue state and rate-limit counters across API instances. Redis must be available at startup.
BullMQ 5	src/jobs/queues.js, src/jobs/overdue.processor.js, src/workers/index.js	Schedules and retries background jobs without delaying HTTP requests; deterministic IDs and a database unique key deduplicate delivery.
Helmet + CORS + cookie-parser	src/app.js, src/middleware/security.js	Adds security headers, exact-origin browser access rules, and cookie handling. Cookie actions require an explicit CSRF header.
express-rate-limit + RedisStore	src/middleware/security.js	Applies separate authentication and API request limits; shared counters work across multiple API processes.
Pino + pino-http	src/config/logger.js, src/app.js	Produces structured logs with request IDs; redaction and sanitized errors avoid exposing authorization and cookie values.
OpenAPI + Swagger UI	src/docs/openapi.js, src/docs/openapi.json, scripts/check-openapi.js	Documents 40 API operations and their schemas; provides an interactive local reference at /api/docs.
Jest + Supertest	tests/unit/, tests/integration/api.test.js	Verifies validation, permissions, sessions, task changes, rollback, and queue behavior through executable checks.
PGlite + redis-memory-server	scripts/test-embedded.mjs	Starts a disposable PostgreSQL-WASM database and real Redis for local verification without using your Supabase credentials.
Docker Compose	compose.yaml, Dockerfile, .dockerignore	Describes local PostgreSQL, Redis, and optional API/worker containers for repeatable setup. No image was deployed here.
ESLint + Prettier + GitHub Actions	eslint.config.js, package.json format script, .github/workflows/ci.yml	Keeps code consistent and defines automated checks against native PostgreSQL 16 and Redis 7. Remote CI has not been run.

Technology	Where it is used	What it does and why it helps
Nodemailer SMTP	src/integrations/email.js	Sends plain-text notification emails with required TLS, bounded network timeouts and a stable Message-ID for correlation.
OneSignal Web Push	src/integrations/onesignal.js	Targets subscribed browsers using a private per-user alias. Uses the outbox UUID to deduplicate provider retries and records no-subscriber outcomes.
Socket.IO	src/realtime/server.js, examples/realtime-client.ts	Authenticates live connections, saves comments, and signals authorized clients to refetch comments and notifications.
PostgreSQL delivery outbox	src/jobs/outbox.repository.js, src/jobs/outbox.processor.js	Atomically saves delivery work, leases pending rows, retries failures and records outcomes independently per channel.

3. How permissions are managed

There are three fixed roles, stored in the `Role` table. An Admin assigns a role when creating a user or changes it through the role endpoint. Custom role creation and individual permission toggles are not implemented. Read the current role descriptions with `GET /api/v1/roles` as Admin.

Operation	Admin	Manager	Employee
Create/edit users, assign roles, deactivate accounts	Yes	No	No
List users	All	Self + members of managed teams	No list; own detail only
Create/edit teams and memberships	Yes	No	No
Read teams	All	Managed teams	Joined teams
Create/edit projects	All teams	Managed teams	No
Read projects	All	Managed teams	Projects with own assigned tasks
Create/edit/delete tasks	All	Managed teams	No
Read tasks, comments, history, work logs	All	Managed teams	Assigned tasks
Change status	Full allowed workflow	Full allowed workflow in scope	Start, block, resume, submit for review
Approve completion/cancel/reopen	Yes	Managed teams	No
Dashboard	Global	Managed teams	Assigned work
Reports	Global	Managed teams	No

Operation	Admin	Manager	Employee
Audit logs	Read only	No	No
Notifications	Own	Own	Own

Authentication first verifies the signed token, then reads the current user and session. It does not trust a role supplied by the browser. Routes apply action-level checks; service queries and mutations apply record-level scopes. For a Manager, `Team.managerId` establishes authority. `User.managerId` is the reporting relationship and does not independently grant team access.

Role changes, password changes, and deactivation invalidate the affected user's sessions. Admins cannot deactivate or demote themselves through these endpoints. A Manager with managed teams or direct reports must be reassigned before demotion/deactivation. Active task assignments prevent removing the corresponding team membership.

4. Authentication lifecycle

1. The client posts email/password to `/auth/login` with `X-CSRF-Protection: 1`.
2. `bcrypt` verifies the password. The API returns a short-lived access JWT and sets an `HttpOnly` refresh cookie.
3. The frontend keeps the access token in memory and sends it as `Authorization: Bearer ....`
4. `/auth/refresh` atomically consumes the current refresh session and creates its replacement. The original family expiry is preserved, so refreshing does not extend a session indefinitely.
5. Reusing an already consumed token revokes the entire family. This revocation commits before the error is returned.
6. Logout revokes the current refresh session. Access-token verification consults that session, so the corresponding access token also stops working.

The refresh cookie defaults to a session cookie unless `Remember Me` is selected. Both choices still have a server-side expiry. HTTPS cookies are required in production. The sample frontend adapter coordinates refresh within one tab; coordinate across tabs before connecting a multi-tab UI.

5. Data and transaction design

User joins `Team` through `TeamMember`, allowing multiple team memberships. A `Project` belongs to one team. A `Task` belongs to a project, stores its team explicitly, and has one assignee. Assignments require an active member of the project's team. A task also has comments, status-history rows, and work activities.

All task modifications carry `version`. For example, two clients both read version 4. The first update changes it to 5. The second request with version 4 receives `409 TASK_VERSION_CONFLICT` and must refetch. The frontend should let the user reconcile the change rather than blindly overwriting it.

Task status changes insert history, an audit event, and an assignee notification inside the same transaction. A failed audit write rolls back the task change. Work-log creation increments actual hours and the task version atomically. Work logs use UTC dates, reject future dates, and limit a user's total logged time to 24 hours per day. PostgreSQL Decimal fields avoid storing hours as floating-point database values.

Task deletion is soft deletion. Audit data is append-only through the API and a SQL trigger rejects normal `UPDATE/DELETE` attempts. This is not protection against a database administrator capable of disabling

triggers or changing the schema.

6. Background notifications

Run `npm run worker` separately from the API. It registers the `daily-overdue` scheduler, defaulting to 08:00 in Asia/Karachi. The scan loads overdue tasks in batches of 250, then adds one notification job per task and assignee. Jobs retry up to five attempts with exponential backoff.

The delivery worker rechecks that the task is still overdue, assigned to the same active user, and not closed or deleted. It upserts a notification using a unique day/task/user deduplication key. Redis job deduplication reduces repeated work; the PostgreSQL unique key is the durable guard against duplicate records. This provides idempotent record creation with at-least-once processing, not a claim of exactly-once execution. Notifications are in-app records with optional Nodemailer email and OneSignal push deliveries. SMS is not included.

Email, push and live delivery work is persisted in `OutboxEvent` rows in the same transaction as notifications/comments. Live and external deliveries have separate queues: an unavailable SMTP server cannot block comment signals. Workers drain every two seconds by default. Leases recover interrupted work and transient failures retry up to eight times. SMTP can still duplicate after an ambiguous provider acceptance; a stable Message-ID is not an exactly-once guarantee.

Email settings are `EMAIL_ENABLED`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS` and `EMAIL_FROM`. OneSignal uses `ONESIGNAL_ENABLED`, `ONESIGNAL_APP_ID`, `ONESIGNAL_REST_API_KEY` and a separate `ONESIGNAL_ID_SECRET`. Defaults are disabled placeholders. Configure real settings and restart both API and worker. See `NOTIFICATIONS-AND-REALTIME.md` for the complete procedure, port/TLS mapping and frontend examples.

OneSignal requires a configured Web Push app, the frontend SDK/service worker, user consent and a browser subscription. Keys alone cannot register a browser. `GET /notifications/push-config` returns only the authenticated user's private alias and public App ID. Push text is generic; confidential details stay behind API authentication. The provider's current identity verification is mobile-only, so the Web integration does not enable that feature.

Socket.IO supports `task:subscribe`, `task:unsubscribe` and `comment:create` with acknowledgements. The server sends `comment:created` and `notification:created` invalidation events. Clients refetch authorized REST data and refetch again on reconnect because Redis Pub/Sub does not retain offline deliveries. Sessions and task access are rechecked during socket operations and fanout. Tokens expire and sockets must reconnect with a refreshed access token.

Comments notify the assignee and team manager, excluding the author and duplicate recipients. An optional `clientRequestId` UUID allows safe retry of the same comment after an acknowledgement timeout; changed content using that ID is rejected. Comment creation, audit, notifications and outbox rows share one transaction. Run `outbox:status` to inspect failures and `outbox:retry` with a failed event UUID after fixing configuration.

7. Where to change things

Desired change	Start here	Important related change
Add a field to tasks	<code>prisma/schema.prisma</code> , <code>task schema/service/repository</code>	Create a migration and update OpenAPI plus frontend mappings
Change role access	<code>src/middleware/authorize.js</code> , the <code>feature route/service</code>	Update cross-role tests and permission documentation
Change workflow statuses	<code>src/modules/tasks/task.constant.js</code> , <code>Prisma enum</code>	Add migration, transition tests, OpenAPI and UI status changes
Change token duration	<code>.env: JWT_ACCESS_EXPIRES_IN</code> , <code>REFRESH_TOKEN_DAYS</code>	Keep frontend refresh and session UX consistent
Change daily reminder time	<code>.env: OVERDUE_CRON</code> , <code>JOB_TIMEZONE</code>	Restart the worker so it updates its scheduler
Add a notification channel	<code>src/integrations/</code> , <code>src/jobs/outbox.repository.js</code>	Add provider credentials, retry policy, and delivery deduplication
Add a report	<code>src/modules/dashboard/dashboard.repository.js</code>	Preserve the role's database filter on every aggregation
Add an API feature	A new folder under <code>src/modules/</code>	Mount in <code>src/app.js</code> ; add schemas, scope checks, tests, OpenAPI

8. Run, verify, and integrate

Follow `README.md` for environment setup, migration, seed, API, and worker commands. Use `API-WALKTHROUGH.md` for requests. Set your own database URLs, Redis URL, JWT secret, allowed frontend origin, and seed password. Do not place server secrets in frontend environment variables.

Local validation is recorded in `VERIFICATION.md`. The embedded database test serializes execution and cannot establish native PostgreSQL multi-connection isolation behavior. Run the included CI checks with your actual PostgreSQL/Redis versions before relying on that behavior. Supabase connectivity, TLS, backups, real DNS/proxy settings, and deployment are your environment's setup steps.

References

- Prisma/Supabase: <https://www.prisma.io/docs/orm/v6/overview/databases/supabase>
- BullMQ scheduling: <https://docs.bullmq.io/guide/job-schedulers/>
- BullMQ idempotent jobs: <https://docs.bullmq.io/patterns/idempotent-jobs>
- PGlite socket test database: <https://pglite.dev/docs/pglite-socket>

The source code and lockfile are the authoritative description of this delivered package. The above links are supporting technology references, not a claim that future versions are compatible without review.

- Nodemailer: <https://nodemailer.com/smtp>
- OneSignal push: <https://documentation.onesignal.com/reference/push-notification>
- OneSignal Web SDK: <https://documentation.onesignal.com/docs/en/web-sdk-reference>
- OneSignal identity verification: <https://documentation.onesignal.com/docs/en/identity-verification>
- Socket.IO middleware: <https://socket.io/docs/v4/middlewares/>